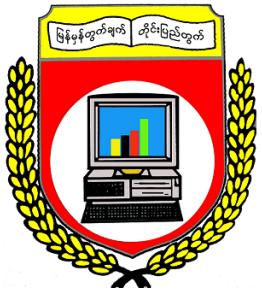


Chapter - 2
Django URLs and Views

University of Computer Studies, Yangon

Outlines

- Django URL Mapping
- Url Parameters, Extra Options, and Query Strings
- View Method Requests
- View Method Responses
- View Method Middleware
- Middleware Flash Messages in View Methods
- Class-Based Views

What is URL?

- A URL is a web address and a URL every time you visit a website – it is visible in browser's address bar.
- **127.0.0.1:8000** is a URL!
- And https://djangogirls.org is also a URL.
- Every page on the Internet needs its own URL.

Django URL Mapping

- Django controller takes over to look for the corresponding view via the url.py file, and then return the HTML response.
- In urls.py, the most important thing is the "urlpatterns".
- It is where you define the mapping between URLs and view.

Django URL Mapping (Cont’d.)

```
from django.views.generic import TemplateView
urlpatterns = [
        url(r'^about/index/',TemplateView.as_view(template_name='index.html')),
        url(r'^about/',TemplateView.as_view(template_name='about.html’)),
]
```

- The "ImportError: cannot import name 'url' from 'django.conf.urls'" occurs because django.conf.urls.url() has been deprecated and removed in version 4 of Django.
- To solve the error, import and use the re_path() method instead.

Django Url Mapping (Cont’d.)

```
from django.urls import re_path, path
from django.contrib import admin
from django.views.generic import TemplateView

urlpatterns = [
        path('admin/', admin.site.urls),
        re_path(r'^about/index/',TemplateView.as_view(template_name='index.html’)),
                re_path(r'^about/',TemplateView.as_view(template_name='about.html’)),
]
```

Django - URL Functions

- The django.urls module contains various functions, path(route, view, kwargs, name) is one of those which is used to map the URL and call the specified view.

Name	Description	Example
path(route, view, kwargs=None, name=None)	It returns an element for inclusion in urlpatterns.	path('index/', views.index, name='main-view')
re_path(route, view, kwargs=None, name=None)	It returns an element for inclusion in urlpatterns.	re_path(r'^index/$', views.index, name='index')

Example - URL Mapping

```
from django.urls import path
from django.views.generic import TemplateView
from app_name import views


urlpatterns = [
        path('', TemplateView.as_view(template_name='home.html')),
        path('index/', views.index),
 ]
```

urls.py

```
import datetime
from django.http import HttpResponse

def index(request):
     now = datetime.datetime.now()
     html = "<html><body><h3>Now time is %s.</h3></body></html>" % now
     return HttpResponse(html)    # rendering the template in HttpResponse
```

views.py

Example - URL Mapping (Cont'd.)

```
from django.urls import path
from app_name import views

urlpatterns = [
        path('books/<int:pk>/', views.book_detail),
        path('books/<str:genre>/', views.books_by_genre),
        path('books/', views.book_index),
 ]
```

- A URL request to **/books/crime/** will match with the second URL pattern. As a result, Django will call the function views.books_by_genre(request, genre = "**crime**").
- Similarly a URL request **/books/25/** will match the first URL pattern and Django will call the function views.book_detail(request, pk =**25**).
- Application name : myapp22

Example –myapp/urls.py

```
from django.urls import path
from . import views

urlpatterns = [
        path('books/<int:pk>/', views.book_detail),
        path('books/<str:genre>/', views.books_by_genre),
        path('books/', views.book_index),
        path('', views.book),
]
```

Example –myapp/views.py

```
from django.shortcuts import render
from django.http import HttpResponse

def book_detail(request,pk=25):
    return HttpResponse("This is book_detail")

def books_by_genre(request,genre="crime"):
    return HttpResponse("This is books_by_genre")

def book_index(request):
    return HttpResponse("This is book_index")

def book(request):
    return HttpResponse("This is book_page")
```

Example –mydjangoproject/urls.py

```
from django.contrib import admin
from django.urls import path, include

urlpatterns = [
        path('admin/', admin.site.urls),
        path('', include('myapp4.urls')),
]
```

Path Converters

- **int** – Matches zero or any positive integer.
- **str** – Matches any non-empty string, excluding the path separator(‘/’).
- **slug** – Matches any slug string, i.e. **a string consisting of alphabets, digits, hyphen and under score.**
- **path** – Matches any non-empty string including the path separator(‘/’)

Using Regular Expressions

- If the **paths and converters syntax** isn't sufficient for defining your URL patterns, you can also use regular expressions. To do so, use re_path() instead of path().
- In Python regular expressions, the syntax for named regular expression groups is (?P<name>pattern), where name is the **name** of the group and **pattern** is some pattern to match.

Example

- **(?P<year>[0-9]{4})**
- re_path(r"^articles/(?P<year>[0-9]{4})/$", views.year_archive),

Regular Expressions Syntax

^ (Start of url)	$ (End of url)	\ (Escape for interpreted values)	| (Or)
+ (1 or more occurrences)	? (0 or 1 occurrences)	{n} (n occurrences)	{n,m} (Between n and m occurrences)
[] (Character grouping)	(?P<name>___) (Capture occurrence that matches regexp ___ and assign it to name	. (Any character)	\d+ (One or more digits). Note escape, without escape matches 'd+' literally.
\D+ (One or more non-digits).Note escape, without escape matches 'D+' literally]	[a-zA-Z0-9_]+ (One or more word characters, letter lower or uppercase, number, or underscore)	\w+ (One or more word characters, equivalent to [a-zA-Z0-9_]). Note escape, without escape matches 'w+' literally].	[-@\w]+ (One or more word character, dash. or at sign). Note no escape for \w since it's enclosed in brackets (i.e., a grouping).

Example

```
from django.urls import path
from app_name import views
urlpatterns =  [
        path("articles/2003/", views.special_case_2003),
        path("articles/<int:year>/", views.year_archive),
        path("articles/<int:year>/<int:month>/", views.month_archive),
        path("articles/<int:year>/<int:month>/<slug:slug>/", views.article_detail), ]
```

```
from django.urls import path, re_path
from app_name import views
urlpatterns = [
        path("articles/2003/", views.special_case_2003),
        re_path(r"^articles/(?P<year>[0-9]{4})/$", views.year_archive),
        re_path(r"^articles/(?P<year>[0-9]{4})/(?P<month>[0-9]{2})/$", views.month_archive),
        re_path( r"^articles/(?P<year>[0-9]{4})/(?P<month>[0-9]{2})/(?P<slug>[\w-]+)/$", views.article_detail, ), ]
```

Regular Expressions Example

- re_path(r'^articles/**(?P<year>[0-9]{4})**/$', views.year_archive
- URL => articles/2026/

- Result =>

  Articles from the year: 2026

Regular Expressions Example

- re_path(
  r"^articles/(?P<year>[0-9]{4})/"
  r"(?P<month>[0-9]{2})/"
  r"(?P<slug>[\w-]+)/$",
  views.article_detail,
)
- URL Example

127.0.0.1:8000/articles/2025/12/python-django-basics/

- Result =>

Details : Year : 2025, Month : 12, Details : python-django-basics

Specifying defaults for view arguments

```
from django.urls import path
from app_name import views
 urlpatterns = [
    path("blog/", views.page),
    path("blog/page<int:num>/", views.page),
]
```

```
def page(request, num=1):

    html = "<html><body><h3>Number of Pages : %s.</h3></body></html>" % num
    return HttpResponse(html)
```

- If the first pattern matches, the page() function will use its default argument for num, 1.
- If the second pattern matches, page() will use whatever num value was captured.

Including other URLconfs

- A function that takes a full Python import path to another URLconf module that should be "included" in this place.
- Next, we add a URL pattern and pass in a route, and the include() function.
- The route is **an empty string** to indicate the **home page**, meaning all other routes that are imported to this pattern are going to start from **the home page**.

```
from django.contrib import admin
from django.urls import path, include #new

urlpatterns = [
    path('admin/', admin.site.urls),
    path('', include('blog.urls')) #new
]
```

Including other URLconfs (Cont’d.)

- This means that all URL routes from the blog app are going to start with the blog/ part then followed by anything you specify in the app urls.py itself.
- Next, we pass in the include() function as the second argument and we pass to it the URLs of our app in the form 'app.urls'. It has to be a string.

```
...
path('blog/', include('blog.urls')) #new
...
```

Passing extra options to view functions

- The path() function can take an **optional third argument which should be a dictionary of extra keyword arguments to pass to the view function**.

```
 urlpatterns = [
    path("detail/", views.detail, {'location':'headquarters'}),
]
```

```
def detail(request,store_id='1',location=None):

     return render(request,'detail.html')
```

Passing extra options to **include()**

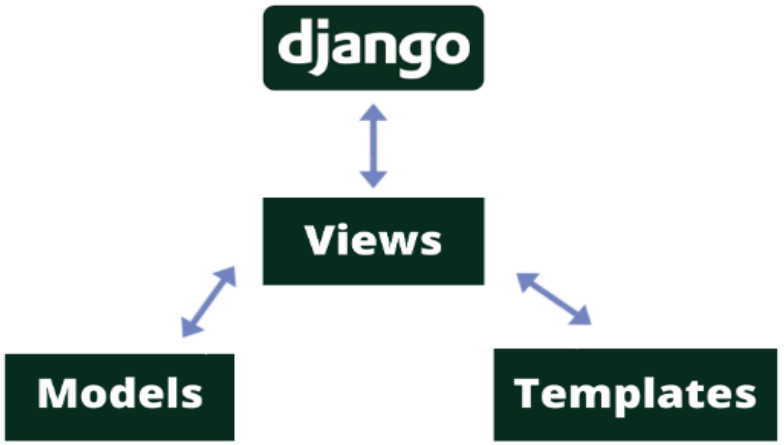
```python
from django.urls import include, path

urlpatterns = [
    path("blog/", include("inner"), {"blog_id": 3}),
]

# inner.py
from django.urls import path
from mysite import views

urlpatterns = [
    path("archive/", views.archive),
    path("about/", views.about),
]
```

Django Views



- A request in Django first comes to **urls.py** and then goes to the matching function in **views.py**.
- Python functions in views.py take the web request from urls.py and give the web response to templates.
- It may go to the data access layer in models.py as per the queryset.
- MVT (Model View Template) where:
  - **Model** is the data access layer,
  - **View** is the business logic layer and
  - **Template** is the presentation layer

Django Views

- A view is a place where we put our business logic of the application.
- The view is a python function which is used to perform some business logic and return a response to the user.
- Views are usually put in a file called views.py

View Method Requests

- **django.http.request.HttpRequest** class contains information set by entities present before a view method: a user's web browser, the web server that runs the application, or a Django middleware class configured on the application.

Django View Method

- **request.GET** or **request.POST** Contains parameters added as part of a GET or POST request, respectively.
- `request.GET.get` is used to extract parameters from an HTTP GET request
- `request.POST.get` to extract parameters from an HTTP POST request

Django View Method

- In Django view method in views.py

```
def detail(request, store_id='1', location=None):

    hours = request.GET.get('hours', ‘’)  #request.POST.get('name',default=None)
    map = request.GET.get('map', '')
       return render(request, 'detail.html')
```

- In the syntax request.GET.get(<parameter>, ''), if the parameter is present in request.GET it **extracts the value** and **assigns it to a variable** for further usage.
- if the parameter is not present then the parameter variable is assigned a **default empty value** of '' . It could be equally used None or any other default value.

View Method Requests(Cont'd)

- request.POST.get('name', default = None) – Gets the value of the name parameter in a POST request.
- request.GET.getlist('drink', default = None) – Gets **a list of values** for the **drink** parameter in a GET request or **gets an empty list** None if the parameter is not present.
- request.META – Contains HTTP headers added by browsers or a web server as part of the request. Parameters are enclosed in a standard Python dictionary where keys are the HTTP header names – in uppercase and underscore (e.g., Content-Length as key CONTENT_LENGTH).

  request.META['REMOTE_ADDR'].- Gets a user's remote IP address.
- request.user – Contains information about a Django user (e.g., username, email) linked to the request.

Set up dictionary in Django View Method

In views.py

```
def detail(request,store_id='1',location=None):

    store_name = 'Downtown'
    store_address = {'street': 'Main #385', 'city': 'San Diego', 'state': 'CA'}
    store_facility = ['WiFi', 'A/C']
    store_menu = ((0, ''), (1, 'Drinks'), (2, 'Food'), (3, 'Coffee'))
    values_for_template = {'store_name': store_name, 'store_address': store_address,
                           'store_facility': store_facility, 'store_menu': store_menu}

    return render(request, 'detail.html', values_for_template)
```

Set up dictionary in Django View Method (Cont'd.)

In template , detail.html

```
<h4> {{store_name}} store </h4>
<p> {{store_address.street}} </p>
<p> {{store_address.city}}, {{store_address.state}} </p>
<hr/>
<p> We offer: {{store_facility.0}} and {{store_facility.1}} </p>
<p> Menu includes : {{store_menu.1.1}} , {{store_menu.2.1}} and {{store_menu.3.1}} </p>
```

Set up dictionary in Django View Method

- the render method includes **the values_for_template** dictionary
- the render method just included the request object and a template to handle the request.
- A dictionary is passed as the last render argument.
- By specifying a dictionary as the last argument, the dictionary becomes available to the template - which in this case is detail.html.
- The dictionary contains keys and values that are data structures declared in the method body. The dictionary keys become references to access the values inside Django templates.

Output View Method Dictionary in Django Templates

<h4>{{store_name}} store</h4>

** {{store_address.street}} **

** {{store_address.city}},{{store_address.state}} <hr/>**

** We offer: {{store_facility.0}} and {{store_facility.1}} **

** Menu includes : {{store_menu.1.1}} and {{store_menu.2.1}} **

- first declaration ***{{store_name}}*** uses the stand-alone key to display Downtown value.
- the other access declarations use dot(.) notation because the values themselves are composite data structures.

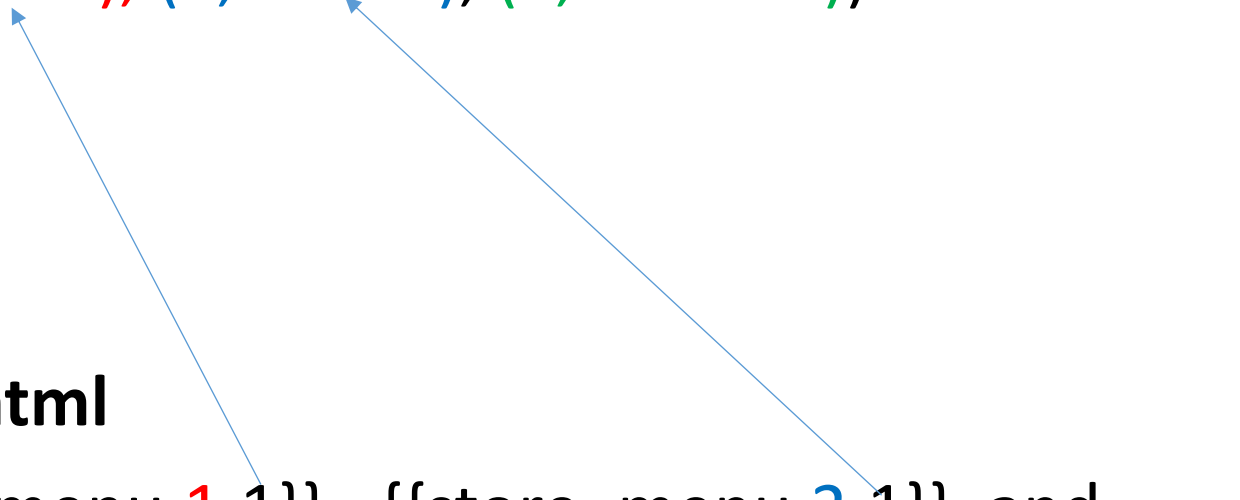
(Cont’d)

- ***store_address*** key contains a dictionary, so to access the internal dictionary values you use the internal dictionary key separated by a dot(.).
- ***store_address.street*** displays the street value, ***store_address.city*** displays the city value, and ***store_address.state*** displays the state value.
- ***store_facility*** key contains a list that uses a similar dot(.) notation to access internal values.
- ***store_menu*** contains a tuple of tuples that also requires a number on account of the lack of keys.
- ***{{store_menu.1.1}}*** displays the second tuple value of the second tuple value of store_menu and ***{{store_menu.2.1}}*** displays the second tuple value of the third tuple of store_menu.

(Cont’d)

myapp4/views.py

store_menu = ((0, ''), (1, 'Drinks'), (2, 'Food'), (3, 'Coffee’))



myapp4 /templates/detail.html

 Menu incluses : {{store_menu.1.1}} , {{store_menu.2.1}} and {{store_menu.3.1}}

View Method Responses

- There are other alternatives to the render() method to generate a view response. But render() method is the most common technique.
- **Three alternatives** of Django view method response.

View Method Response(Option1)

```
from django.shortcuts import render

def detail(request,store_id='1',location=None):
     ...
    return render(request,'stores/detail.html', values_for_template)
```

- django.shortcuts.render() method that shows three arguments to generate a response: the (required) request reference, a (required) template route and an (optional) dictionary – also known as the context – with data to pass to the template.

Example (Option 1)

```
from django.shortcuts import render


def detail(request,store_id='1',location=None):
    store_name = 'Downtown'
    store_address = {'street': 'Main #385', 'city': 'San Diego', 'state': 'CA'}
    store_facility = ['WiFi', 'A/C']
    store_menu = ((0, ''), (1, 'Drinks'), (2, 'Food'), (3, 'Coffee'))
    values_for_template = {'store_name': store_name, 'store_address': store_address,
            'store_facility': store_facility, 'store_menu': store_menu}
    return render(request, 'detail.html', values_for_template)
```

View Method Response(Option2)

```
from django.template.response import TemplateResponse
def detail(request,store_id='1',location=None):
...
return TemplateResponse(request, 'stores/detail.html', values_for_template)
```

- django.template.response.TemplateResponse() class, which is nearly identical to the render() method.
- The difference between the two variations is that TemplateResponse() can alter a response once a view method is finished (e.g., via middleware), where as the render() method is considered the last step in the life cycle after a view method finishes.

Example (Option 2)

```
from django.template.response import TemplateResponse


def detail(request,store_id='1',location=None):
    store_name = 'Downtown'
    store_address = {'street': 'Main #385', 'city': 'San Diego', 'state': 'CA'}
    store_facility = ['WiFi', 'A/C']
    store_menu = ((0, ''), (1, 'Drinks'), (2, 'Food'), (3, 'Coffee'))
    values_for_template = {'store_name': store_name, 'store_address': store_address,
            'store_facility': store_facility, 'store_menu': store_menu}
    return TemplateResponse(request, 'detail.html', values_for_template)
```

View Method Response(Option3)

```
from django.http import HttpResponse
from django.template import loader, Context

def detail(request,store_id='1',location=None):
    ...
    response = HttpResponse()
    t = loader.get_template('stores/detail.html’)
    c = Context(values_for_template)
    return response.write(t.render(c))
```

- This process first creates a raw HTTPResponse instance, then loads a template with the django.
- template.loader.get_template() method, creates a Context() class to load values into the template, and finally writes a rendered template with its context to the HTTPResponse instance.

Example (Option 3)

```
from django.http import HttpResponse
from django.template import loader, Context


def detail(request,store_id='1',location=None):
    store_name = 'Downtown'
    store_address = {'street': 'Main #385', 'city': 'San Diego', 'state': 'CA'}
    store_facility = ['WiFi', 'A/C']
    store_menu = ((0, ''), (1, 'Drinks'), (2, 'Food'), (3, 'Coffee'))
    values_for_template = {'store_name': store_name, 'store_address': store_address,
            'store_facility': store_facility, 'store_menu': store_menu}


    # edited on old version
    template=  loader.get_template('detail.html')
    eturn HttpResponse(template.render(values_for_template,request))
```

HTTP Status and Content-Type Headers

- There are two type of response options: HTTP Status and HTTP Content-Type
- **The HTTP Status header** is a three-digit code number to indicate the response status for a given request.

HTTP status code	Python code sample
404 (Not Found)	from django.http import Http404 raise Http404
500 (Internal Server Error)	raise Exception
400 (Bad Request)	from django.core.exceptions import SuspiciousOperation raise SuspiciousOperation
403 (Forbidden)	from django.core.exceptions import PermissionDenied raise PermissionDenied

HTTP Status and Content-Type Headers

- **The HTTP Content-Type header** is a **MIME** (Multipurpose Internet Mail Extensions) type string to indicate **the type of content** in a **response**.
- Examples –
  - Content-Type values are text**/html**, which is the standard for an **HTML** content response
  - image/**gif**, which is used to indicate a response is a **GIF image**.

HTTP Content-type and HTTP Status for Django view method responses

return render(request, 'stores/menu.csv', values_for_template, content_type='text/plain')

- In this example, return **a response** with plain text content. Notice the render method content_type argument.

return render(request,'stores/menu.json', values_for_template, content_type='application/json')

- In this example, return with JavaScript Object Notation(JSON) content.

HTTP Content-type and HTTP Status for Django view method responses(Cont’d)

return render(request,'custom/notfound.html',status=404)

return render(request,'custom/internalerror.html',status=500)

- The examples set the HTTP Status code to 404 and 500.
- Because the HTTP Status **404** code is used **for resources that are not found**, the render method uses a special template for this purpose.

View Method Middleware

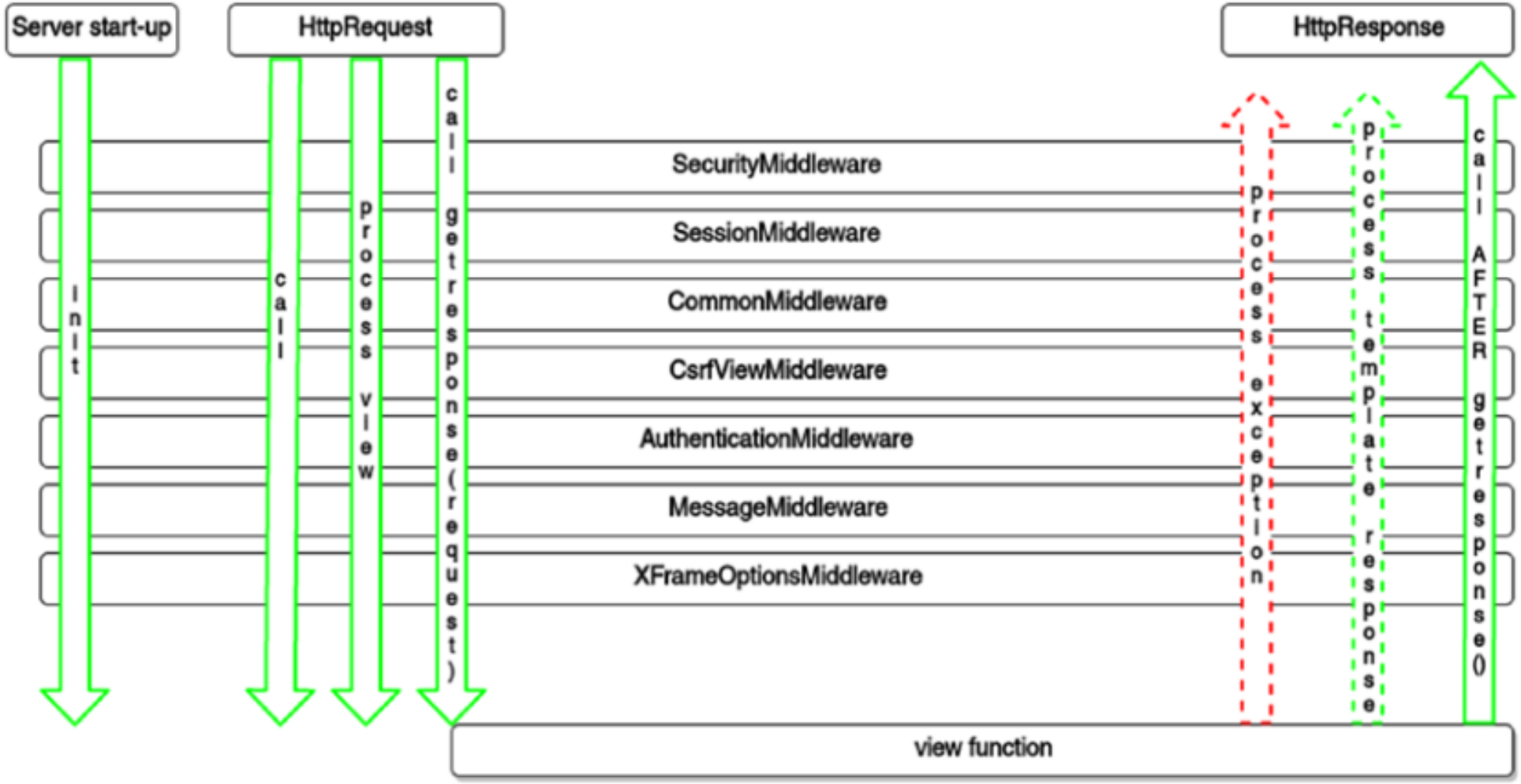
- **Middleware** is a framework of hooks into Django's request/response processing.
- It's a light, low-level "plugin" system for globally altering Django's input or output.
- Each middleware component is responsible for doing some specific function. The functions can be a **security, session, csrf protection, authentication** etc.
- Open a Django project's settings.py file , the MIDDLEWARE variable whose default contents are shown in the following:

View Method Middleware

```
MIDDLEWARE = [
    'django.middleware.security.SecurityMiddleware',
    'django.contrib.sessions.middleware.SessionMiddleware',
    'django.middleware.common.CommonMiddleware',
    'django.middleware.csrf.CsrfViewMiddleware',
    'django.contrib.auth.middleware.AuthenticationMiddleware',
    'django.contrib.messages.middleware.MessageMiddleware',
    'django.middleware.clickjacking.XFrameOptionsMiddleware',
]
```

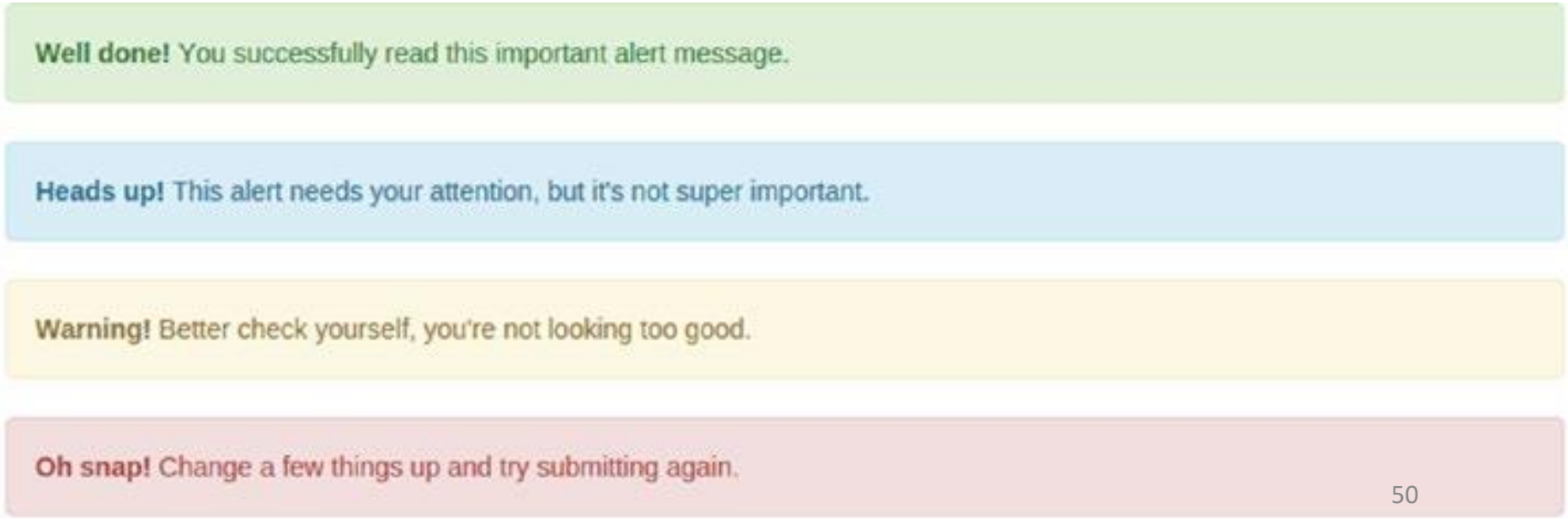
- Django projects enabled with **seven middleware classes**, so all requests and responses are set to run through these seven classes.

Middleware Structure and Execution Process



Middleware Flash Messages in View Methods

- Flash messages are typically used when users perform an action (e.g., submit a form)
- It’s necessary to tell them if the action was successful or if there was some kind of error.
- Moreover, flash messages are also **used as one-time notifications on web pages** to tell users about certain events.



Middleware Flash Messages in View Methods

- By default, all Django projects are enabled to support flash messages.
- Flash messages require a **django app**, **middleware**, and a **template context processor**
- In order to work Django flash messages, the following values are set in settings.py:
  - the variable INSTALLED_APPS has the **django.contrib.messages** value,
  - the variable MIDDLEWARE has the **django.contrib.messages.middleware.MessageMiddleware** value,
  - and the context_processors list in options of the templates variable has the **django.contrib.messages.context_processors.messages** value.

Built-in Flash Messages

Level Constant	Tag	Value	Purpose
DEBUG	debug	10	Development-related messages that will be ignored (or removed) in a production deployment.
INFO	info	20	Informational messages for the user.
SUCCESS	success	25	An action was successful, for example, "Contact info was sent successfully."
WARNING	warning	30	A failure did not occur but may be imminent.
ERROR	error	40	An action was not successful or some other failure occurred.

Add Flash Messages

- To add messages you use the django.contrib.messages package.
- There are two techniques to add flash messages with the django.contrib.messages package: one is the generic **add_message()** method, and the other is shortcuts methods for the different levels.

Generic add_message method (option 1)

```
from django.contrib import messages

messages.add_message(request, messages.DEBUG, 'The following SQL statements were executed:

%s' % sqlqueries)

messages.add_message(request, messages.INFO, 'All items on this page have free shipping.')

messages.add_message(request, messages.SUCCESS, 'Email sent successfully.')

messages.add_message(request, messages.WARNING, 'You will need to change your password in

one week.')

messages.add_message(request, messages.ERROR, 'We could not process your request at this time.')
```

Shortcut level methods (option 2)

***messages.debug** (request , 'The following SQL statements were executed: %s' % sqlqueries)*

***messages.info** (request , 'All items on this page have free shipping.')*

***messages.success** (request , 'Email sent successfully.')*

***messages.warning** (request , 'You will need to change your password in one week.')*

***messages.error** (request , 'We could not process your request at this time.')*

Example – Flash Message

```
# urls.py

urlpatterns = [
    path('admin/', admin.site.urls),
    path('welcome/',views.flash_message),
]
```

```
#view.py
def flash_message(request):
    data = dict()
    messages.success(request, "Success: This is the sample success Flash message.")
    messages.error(request, "Error: This is the sample error Flash message.")
    messages.info(request, "Info: This is the sample info Flash message.")
    messages.warning(request, "Warning: This is the sample warning Flash message.")
    return render(request, 'welcome.html', data)
```

Assess Flash Message

In Template, welcome.html

```
<html>
<head></head>
<body>
{% for message in messages %}
<div class="alert {{ message.tags }} alert-dismissible" role="alert">
    <button type="button" class="close" data-dismiss="alert" aria-label="Close">
        <span aria-hidden="true">&times;</span>
    </button>
    {{ message | safe }}
</div>
{% endfor %}
</body>
</html>
```

Flash Message Example - welcome.html

```
<style>
    .alert {
        padding: 12px;
        margin: 8px 0;
        border-radius: 5px;
        font-size: 16px;
        font-weight: bold;
    }

    .alert-success {
        background:#d4edda;
        color:#155724;
        border: 1px solid#c3e6cb;
    }
```

```
.alert-error, .alert-danger {
        background:#f8d7da;
        color:#721c24;
        border: 1px solid#f5c6cb;
    }

    .alert-info {
        background:#d1ecf1;
        color:#0c5460;
        border: 1px solid#bee5eb;
    }

    .alert-warning {
        background:#fff3cd;
        color:#856404;
        border: 1px solid#ffeeba;
    }
</style>
```

```
<body>
{% if messages %}
    {% for message in messages %}
        <div class="alert alert-{{ message.tags }}">
            {{ message }}
        </div>
    {% endfor %}
{% endif %}
</body>
```

Flash Message Example - myapp/views.py

```
from django.shortcuts import render
from django.contrib import messages

def flash_message(request):
    data = dict()
    messages.success(request, "Success: This is the sample success Flash message.")
    messages.error(request, "Error: This is the sample error Flash message.")
    messages.info(request, "Info: This is the sample info Flash message.")
    messages.warning(request, "Warning: This is the sample warning Flash
message.")
    return render(request, 'welcome.html')
```

Built-in Class-Based Views

- Class-based views provide an alternative way to **implement views as Python objects instead of functions.**
- Object oriented techniques such as mixins (multiple inheritance) can be used to factor code into reusable components.

Example - Built-in Class-Based Views

```python
from django.contrib import admin
from django.urls import path
from storeapp.views import MyView

urlpatterns = [
    path('admin/', admin.site.urls),
    path("about/", MyView.as_view()),
]
```

urls.py

```python
class MyView(View):
    def get(self, request):
        # <view logic>
        return HttpResponse("result")
```

views.py

Built-in Class-Based Views

Class	Description
django.views.generic.View	Parent class of all class-based views, providing core functionality.
django.views.generic.TemplateView	Allows a url to return the contents of a template, without the need of a view.
django.views.generic.RedirectView	Allows a url to perform a redirect, without the need of a view.
django.views.generic.ArchiveIndexView django.views.generic.YearArchiveView django.views.generic.MonthArchiveView django.views.generic.WeekArchiveView django.views.generic.DayArchiveView django.views.generic.TodayArchiveView django.views.generic.DateDetailView	Allows a view to return date-based object results, without the need to explicitly perform Django model queries.
django.views.generic.CreateView django.views.generic.DetailView django.views.generic.UpdateView django.views.generic.DeleteView django.views.generic.ListView django.views.generic.FormView	Allows a view to execute Create-Read-Update-Delete (CRUD) operations , without the need to explicitly perform Django model queries.

Thank You